# Pore-network flow analysis from OpenFOAM DNS

Jose Arnal Trespallé

June 4, 2026

---

A post-processing toolchain that turns a direct numerical simulation (DNS) of single-phase flow through a disordered porous medium (computed with OpenFOAM) into a **directed pore network**, and then studies how flow is distributed across that network using both a physical (Stokes/Poiseuille) model and a topological random-splitting model.

The pipeline goes end to end: from the raw CFD field to network extraction, to stochastic and physical flow models, to the final publication-quality figures.

**Repository:** `https://github.com/trespalle/OpenFOAM_to_PNM`

## Contents

# 1. What the code does

Given a converged OpenFOAM case of creeping flow around an array of grains, the toolchain:

1. **Reconstructs and exports** the latest CFD time step to VTK.
2. **Extracts the pore network** from the velocity and pressure fields: it detects the grains, builds the throats (pore links) between them via a Delaunay triangulation of grain centers, measures each throat's geometry (half-aperture) and the volumetric flow rate through it, orients each throat by the sign of the local CFD velocity, and records the pressure at each junction. The result is a directed graph with measured **splitting fractions** (the share of a junction's outgoing flow carried by each throat).
3. **Runs a topological random-splitting model** on that graph to obtain the statistical distribution of tube flow rates $P(q)$, and fits an analytical prediction to it.
4. **Solves the physical flow problem** on the same network (forward and inverse Stokes problems) and compares the measured/effective conductances against the purely geometric Poiseuille prediction, plus several null models.
5. **Sweeps the geometric disorder** to measure how the normalized variance of the input aperture distribution maps onto the normalized variance of the output flow-rate distribution (variance scaling).
6. **Produces the final figures** from all of the above.

## 1.1 Scientific framing

The physical model assumes **creeping (Stokes) flow** through a thin quasi-2D cell, so that each throat behaves as a hydraulic resistor:

$$Q = \frac{g\,\Delta P}{\mu}, \qquad \text{with geometric conductance} \qquad g = \frac{2Ha^3/3}{L},$$

where $a$ is the throat half-aperture, $L$ the throat length, $H$ the out-of-plane cell thickness and $\mu$ the dynamic viscosity. Mass conservation at each junction closes the linear network, which is solved as a sparse linear system. A `HeleShaw` conductance model (finite-aspect-ratio rectangular duct, solved with a Newton–Raphson inversion of the transcendental cubic law) is also available.

The random-splitting model treats flow propagation as a steady-state mass balance on the directed acyclic graph, where the measured splitting fractions act as transition weights. The analytical tube-flow distribution is built as

$$P(q) = \tfrac{1}{3}\,g(q) + \tfrac{2}{3}\,f(q)$$

from a Gamma distribution fitted to the pore-mass distribution.

# 2. Pipeline architecture

Everything is orchestrated by `run_post.sh`, which takes a single argument (the path to the OpenFOAM case) and runs the following stages in order:

| # | Step | Component | Output |
|---|---|---|---|
| 1 | Reconstruct + export CFD | `reconstructPar`, `foamToVTK` | `VTK/` |
| 2 | Network extraction | `halfwidths_and_fluxes_v6.ipynb` | `junction_links.txt`, `junction_coordinates.txt` |
| 3 | Random-splitting model | `random_splitting_def.cpp` (+ `calculate_fit.py`) | flow/pore distributions, analytical fit |
| 4 | Visualization | `random_splitting_well_plotted_def.ipynb` | figures |
| 5 | Physical flow tests | `pore_tests.cpp` | conductance/aperture histograms, null models |
| 6 | Variance scaling | `variance_scaling.cpp` | $CV^2$-vs-$CV^2$ curves |
| 7 | Final figures | `variance_scaling_well_plotted.ipynb` | publication figures (PDF) |

**Fast mode:** if `junction_links.txt` already exists for a case, Stage 2 (the slow notebook, ∼7 min) is skipped and the pipeline reuses the existing network.

Results for each case are written to `Processed_cases/<case_name>_processed/`, with `raw_data/`, `notebooks/` and `figures/` subfolders.

# 3. Components

## 3.1 Core libraries (C++ headers)

- `graph_theory.h` — base `Graph` class: nodes, links, boundary detection, splitting-fraction renormalization, DAG check, BFS distances, and the steady-state mass solver (`compute_stationary_flows`), built on a sparse linear solve (Eigen `SparseLU`).
- `pore_networks.h` — `PoreNetwork` (derived from `Graph`): physical layer. Geometric/effective conductances, Poiseuille and Hele-Shaw conductance laws, pressure-field solver (`solve_flow_field`), and the forward/inverse Stokes problems.
- `myfunctions.h` / `myfunctions.cpp` — shared numerical utilities used by all three executables: mean/variance estimators with percentile clipping (`med_var`), histogram construction (`buildHistogram`), and histogram/gnuplot output (`guardaHistograma`). **This is a mandatory build dependency** — every C++ program must be compiled and linked against `myfunctions.cpp`.
- `Rand.h` — self-contained RNG with normal and Gamma sampling (Marsaglia–Tsang).

## 3.2 Executables (compiled from `.cpp`)

- `random_splitting_def.cpp` — topological random-splitting model; computes tube-flow and pore-mass distributions and shells out to `calculate_fit.py` and `gnuplot`.
  CLI: `random_splitting<links_file>[n_realizations][n_bins][coords_file]`
- `pore_tests.cpp` — physical inverse/forward problem and null models (geometric vs effective conductance, topological splitting, random weights, shuffled apertures).
  CLI: `pore_tests<output_dir><links_file><coords_file>`
- `variance_scaling.cpp` — sweeps the aperture-disorder parameter and measures input-vs-output normalized variance.
  CLI: `variance_scaling<output_dir><links_file><coords_file>`

### 3.3 Python / notebooks

- `halfwidths_and_fluxes_v6.ipynb` — CFD-to-network extraction.
- `calculate_fit.py` — fits the Gamma distribution and builds the analytical $P(q)$ curve. CLI: `pythoncalculate_fit.py<raw_data_file><output_base_path>`
- `random_splitting_well_plotted_def.ipynb`, `variance_scaling_well_plotted.ipynb` — final figure generation.

## 4. Data formats

The network is passed between stages through two whitespace-separated text files written by the extraction notebook.

`junction_links.txt` — one row per throat (directed `in` → `out`):

```
in_id   out_id   weight   half_width   flow_rate
```

- `weight` — splitting fraction: the share of the source junction's outgoing flow carried by this throat (renormalized to sum to 1 per node by the C++).
- `half_width` — throat half-aperture, in **meters** (SI).
- `flow_rate` — volumetric flow rate through the throat, in $\mathbf{m^3/s}$.

`junction_coordinates.txt` — one row per junction:

```
id   x   y   pressure
```

with `x`, `y` in **meters** and `pressure` in **Pa**.

## 5. Requirements

### 5.1 System tools

- OpenFOAM (provides `reconstructPar`, `foamToVTK`).
- A C++17 compiler (`g++` recommended; the build uses `-O3`).
- Eigen 3.4 or newer (header-only). The build script includes it from a local folder (`eigen-5.0.0/` in the tool directory); adjust the `-I` path in `run_post.sh` to wherever your Eigen headers live.
- gnuplot (used by `random_splitting` to render its fit plots).
- Jupyter / `nbconvert` (to execute the notebooks headlessly).

### 5.2 Python

A virtual environment named `tubes` is expected by the script (see Setup). Tested with Python 3.10+. Required packages:

```
numpy
scipy
matplotlib
pandas
pyvista          # VTK reading
shapely          # grain polygonization
scikit-image     # distance transform, peak detection
mpmath           # high-precision Gamma functions in calculate_fit.py
jupyter
notebook
```

*Note (NumPy 2.0):* the extraction notebook uses `np.trapz`, which is deprecated in NumPy 2.0. If you run on NumPy $\geq$ 2.0, replace it with `np.trapezoid`.

## 6. Setup

```
# 1. Clone
git clone <your-repo-url>
cd <repo>

# 2. Create the Python virtual environment expected by run_post.sh
python3 -m venv tubes
source tubes/bin/activate
pip install -r requirements.txt
deactivate

# 3. Make Eigen available (either install system-wide or drop the headers
#    in a local folder and update the -I path in run_post.sh)
```

## 7. Usage

Run the full pipeline on a converged OpenFOAM case:

```
./run_post.sh /path/to/your/openfoam/case
```

The script reconstructs the latest time step, extracts the network (unless it already exists), compiles the three C++ programs, runs them, and executes the visualization notebooks. All artifacts for the case appear under `Processed_cases/<case_name>_processed/`.

To run an individual stage manually (e.g. after editing the C++), compile the program **together with** `myfunctions.cpp`, which all three executables depend on:

```
EIGEN=/path/to/eigen
RAW=Processed_cases/mycase_processed/raw_data

# random-splitting model
g++ -std=c++17 -O3 -I$EIGEN random_splitting_def.cpp myfunctions.cpp -o random_splitting
./random_splitting $RAW/junction_links.txt 1000 100 $RAW/junction_coordinates.txt

# physical flow tests
g++ -std=c++17 -O3 -I$EIGEN pore_tests.cpp myfunctions.cpp -o pore_tests
./pore_tests $RAW/ $RAW/junction_links.txt $RAW/junction_coordinates.txt

# variance scaling
```

```
g++ -std=c++17 -O3 -I$EIGEN variance_scaling.cpp myfunctions.cpp -o variance_scaling
./variance_scaling $RAW/ $RAW/junction_links.txt $RAW/junction_coordinates.txt
```

## 8. Conventions and assumptions

These are important for reproducing or extending the results:

- **Units are SI throughout.** Coordinates and apertures are in meters (as exported by OpenFOAM), pressures in Pa, flow rates in $m^3/s$.
- **Out-of-plane thickness** $H$ is fixed at `0.001 m` (1 mm) in the conductance laws and in the flow-rate integration in the notebook. Change it consistently in both places if your cell thickness differs.
- **Viscosity.** OpenFOAM (incompressible) specifies a *kinematic* viscosity $\nu$ and works with the kinematic pressure $p/\rho$. The notebook converts pressure to Pa ($\times \rho$), so the consistent *dynamic* viscosity for the physical comparison is $\mu = \nu \cdot \rho$. This value is set in `pore_tests.cpp` (`MU_DNS`); it must match the DNS, not the working fluid. It only affects the absolute geometric-vs-effective conductance comparison; all mean-normalized distributions are invariant to a global $\mu$.
- **Stokes regime.** The network model is linear (Stokes). It is valid only at low Reynolds number; at appreciable $Re$ the effective conductance lumps in inertial losses and the geometric comparison should be read accordingly.

## 9. Output

For each processed case you get, under `Processed_cases/<case>_processed/`:

- `raw_data/` — extracted network (`junction_links.txt`, `junction_coordinates.txt`) and the `.dat` histograms / analytical curves produced by the C++ programs.
- `figures/` — generated plots.
- `notebooks/` — the executed notebooks for that case.

## 10. Project layout

```
.
|-- run_post.sh                          # pipeline orchestrator
|-- graph_theory.h                       # base Graph class
|-- pore_networks.h                      # PoreNetwork physical layer
|-- myfunctions.{h,cpp}                  # numerical utilities
|-- Rand.h                               # RNG
|-- random_splitting_def.cpp             # random-splitting model
|-- pore_tests.cpp                       # physical flow tests + null models
|-- variance_scaling.cpp                 # variance scaling sweep
|-- calculate_fit.py                     # analytical Gamma fit
|-- halfwidths_and_fluxes_v6.ipynb       # CFD → network extraction
|-- random_splitting_well_plotted_def.ipynb
`-- variance_scaling_well_plotted.ipynb
```